FPGA Latch Primitive based Efficient True Random Number Generators

Mustafa Mert Esen, Şükrü Uzun, and Emre Göncü
Secure Information Research (SIR) Team
Procenne, Istanbul, Turkey
{mustafa.esen, sukru.uzun, emre.goncu}@procenne.com

2023 30th IEEE International Conference on Electronics, Circuits and Systems (ICECS) | 979-8-3503-2649-9/23/$31.00 ©2023 IEEE | DOI: 10.1109/ICECS58634.2023.10382907

***Abstract*—In cryptography, true random number generators are essential and have a crucial role. The study provides an innovative and effective way for producing true random numbers on FPGAs by taking advantage of the metastability of latches and the phase noises of ring oscillators. The proposed noise sources are based on Latch primitives provided by the FPGA vendor. Since it uses less Look-up tables (LUTs), using that kind of resource for noise sources is area-efficient. Another key benefit of the suggested true random numbers generator is that it just needs a limited number of primitives and doesn't require any FPGA-specific configuration or constraint before implementation. The proposed TRNGs are validated on three different FPGA parts (XC7Z015, XC7Z020, XC7Z045). Without any post-processing, they pass all AIS-20/31 tests, NIST 800-22 tests, NIST 800-90B tests, and NIST 800-22 tests for all the FPGA ICs. Hence, the proposed TRNGs have the best area-speed efficiency values compared to the other TRNG implementations given in the literature.**

***Index Terms*—TRNG, FPGA, Metastability, Race Condition, Ring Oscillator, Latch based TRNG**

I. Introduction

True Random Number Generator (TRNG) is a necessary component of cryptographic hardware systems to produce unpredictable cryptographic keys for algorithms such as RSA (River-Shamir-Adlemen). RNGs are typically divided into two groups: deterministic (pseudo) random number generators (PRNGs) and nondeterministic (true) random number generators (TRNGs). TRNGs are widely used to seed PRNGs into an arbitrary-length sequence to produce an unpredictable sequence. Many different physical designs are used as entropy sources. TRNGs gather randomness from physical processes, such as temperature, noise, radioactive decay, etc. The most well-known techniques for random number generators such as free-running oscillators, chaos, and metastability are based on physical noises [1].

The main idea behind physical noise-based TRNGs is to sample the physical entropy source by comparing it with a threshold. However, for that to happen, the majority of physical component noises need to be converted to an electrical signal. Several of the common noise sources include: power supply [2], laser phase noise [3], chaos noise [4], ADC sampling [5]. These noises are significantly influenced by environmental factors such as temperature, humidity, etc.

*Secure Information Research Team

There are some straightforward deterministic circuit designs that can produce chaos. Time series generated by the chaotic process appears uncertain to the observer because of the complex dynamic behavior within the brief observation period [6]. Optical [3], [4], [7], electrical, or even mechanical [8] chaotic systems are currently the most practical chaotic systems for generating random numbers.

The phase noise of ring oscillators is a good candidate as a noise source for the TRNGs [9], [10]. A logical inverter circuit transforms into a free-running oscillator (ring oscillator) when its output is connected to the circuit's input. The Zeno paradox is created when its output is connected to its input: if the output is in a logical '1' state, then the input will be '1', and the inverter will cause the output to go '0'. Once the output changes to "0," the inverter action will cause it to change to "1," and so on.

The metastability behaviors of latches are another entropy source for TRNGs [11], [12], [13], [14]. Device mismatches and the thermal noise cause the metastable point to be settled on '1' or '0', randomly. Some systems additionally take advantage of hardware-specific primitives provided by vendors to achieve metastability [15], [16], [17].

In this paper, four different novel TRNG designs are proposed. The three of them are latch based and the last one is implemented with Look-up tables (LUTs). The latch-based designs employ the Xilinx 7-Series Latch primitives, LDPE and LDCE [18]. Although the proposed design uses Xilinx-specific primitives, the designs can be extended to the other FPGA vendors since they have similar primitives. The key benefit of using that kind of resource is not consuming the additional logic resources of the FPGA like LUTs. In addition, all the proposed designs require no FPGA-specific constraints like localization constraints which is the case for most of the metastability-based TRNG implementations on FPGAs [14]. Furthermore, the designs pass all the statistical tests of NIST 800-22 [19], NIST 800-90B [20] and, AIS-20/31 [21] tests, without any post-processing.

The paper is organized as follows. In Section II, brief explanations for metastability and ring-oscillators are given. In Section III, the details of the proposed noise sources are explained. In addition, the TRNG design employing the noise sources is introduced and the experiments for finding the optimum clock frequencies and the number of noise sources are given in the same section. Furthermore, the experimental

setup, the applied statistical tests, and their results are given in Section IV. Finally, the paper is concluded with Section V.

II. Background

Before introducing the proposed designs, the fundamentals of the proposed noise sources, metastability, and phase noise of the ring oscillators are explained. Furthermore, the design challenges of both concepts are discussed.

A. Metastability and Race Condition

The basic schematic of an RS latch-based TRNG is shown in Fig. 1. Two NAND gates or two NOR gates, with each gate's output coupled to one of the other gate's inputs. In this paper, RS latches with NAND gates are considered. Both Q and $\bar{Q}$ become '1' when the `Clk` signal is reset to '0'. The latch goes into a metastable condition during the rising edge of the `Clk` and subsequently goes into one of the stable states i.e. $(Q, \bar{Q}) = (0, 1)$ or $(1, 0)$, randomly.

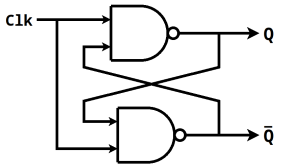


Fig. 1: RS Latch for random number generation.

The output of a real RS latch, however, is biased; it always resolves to '0' or '1'. The bias is caused by two main factors, thermal noise, and device mismatches. Due to the hardness of making the symmetrical lines and components equal, the device mismatch can dominate the thermal noise. Therefore, the system resolves always to the same states. Although the idea behind a metastability-based TRNG is simple, achieving high-quality randomness is challenging, especially in FPGA implementations. Any type of circuit imbalance or device mismatches will result in biasing of the output. For the FPGA implementations, making the symmetrical lines equal is almost not possible. Because of that, a tuning mechanism is employed in [14]. But this method increases resource consumption, for sure.

B. Ring Oscillator

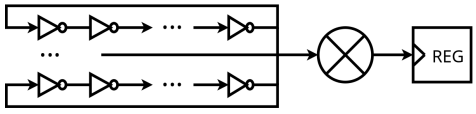


Fig. 2: Simple model of ring oscillator.

The conventional ring oscillator is made up of an odd number of inverters arranged in a ring. However, the most basic ring oscillator model can be depicted by a single inverting transfer function. As in shown Fig. 2, an XOR gate receives the output of a group of identical ring oscillators. The output is then sampled by a register using a system clock to provide true random numbers. The low entropy in this architecture necessitates post-processing, which significantly reduces the throughput rate. Normally, the entropy can be improved by adding more oscillators. But it might lead to higher power consumption.

III. Proposed Designs

Four distinct noise source designs are suggested in this research. Also introduced is the TRNG design, which consists of numerous proposed noise sources. Additionally, a number of analyses are performed using NIST 800-22 statistical test suite's *Non-Overlapping Template* test in order to determine the ideal noise source counts and clock frequencies.

A. Noise Source Designs

1) Latch with XNOR (LWXNOR): Although latch-based TRNGs are based on a straightforward concept, many design-related reasons can affect negatively the random quality of the outputs. If the design-related issues are handled, metastability and race conditions can be a good noise source for TRNGs.

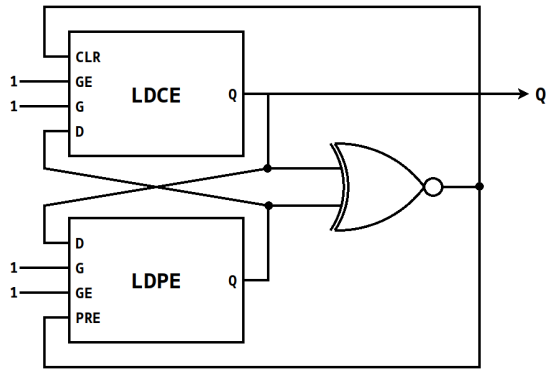


Fig. 3: Proposed design with latches and XNOR gate.

The proposed design in Fig. 3 shows a noise source that can be created using the LDCE and LDPE primitives provided by Xilinx FPGAs. The cell named LDCE stands for "Transparent Latch with Clock Enable and Asynchronous Clear" and LDPE stands for "Transparent Latch with Clock Enable and Asynchronous Preset" [18]. Despite the fact that the underlying

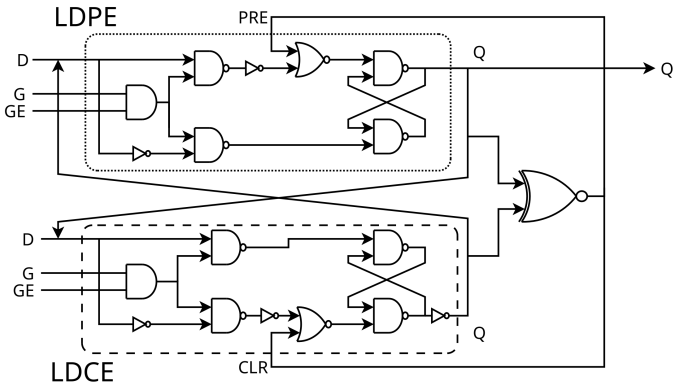


Fig. 4: LWXNOR design modelled with D latches.

structure of these primitives is not provided in the source,

the schematics of a D latch with asynchronous reset and asynchronous preset can be given as in Fig. 4.

In the proposed noise source, G and GE inputs are constantly connected to '1'. In this case, the reduced circuit is given in Fig. 5. Assuming that the Q outputs of LDCE and LDPE are (0,1) in the initial state, the XNOR output is '0', therefore PRE and CLR inputs are (0,0). Thus, metastability and race condition occurs between LDCE and LDPE. The race continues until the Q outputs of LDCE and LDPE are equalized. When the race is over, the outputs of LDCE and LDPE will be equal to (0,0) or (1,1). In both cases, the XNOR output will be '1'. PRE and CLR inputs become '1'. The design returns to its initial state (0,1), and PRE and CLR inputs become '0' again. In this way, metastability and race conditions are made continuous. Each time the race is over, the design returns to its initial state and the race begins again.

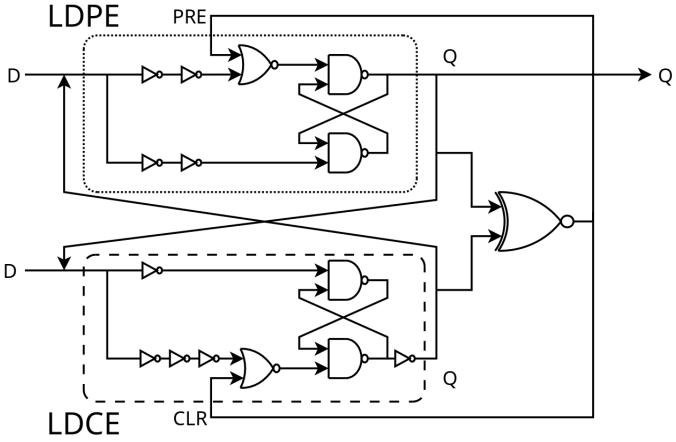


Fig. 5: Reduced LWXNOR design.

The working principle of the noise source is as follows: assuming that the Q outputs of LDCE and LDPE are (0,1) in the initial state.

1) G and GE inputs are set to '1'.
2) XNOR inputs are (0,1).
3) The PRE and CLR inputs are '0'.
4) Metastability and race condition occurs and continues until the outputs are equalized.
5) LDCE and LDPE outputs are (0,0) or (1,1).
6) Inputs of XNOR are (0,0) or (1,1), so output is '1'.
7) The PRE and CLR inputs are '1'.
8) The LDCE output is '0'. The LDPE output is '1'.
9) XNOR inputs are (0,1). So returns to step 2.

Thus, there is no need to provide constraints for delays and placement of cells during designing, since the Metastability and race condition restart over and over again regardless of the delay amount and the placement. The Q output of LDCE or LDPE can be sampled by another register to generate random numbers.

2) LWXNOR with LUT (LWXNORLUT): LWXNORLUT is the noise source designed by using the truth table of the noise source, LWXNOR, given in the previous section. The truth table obtained by the reduced block diagram (Fig. 5) is given in Table I. The truth table is realized by LUT6_2 primitives provided by Xilinx [18].

TABLE I: LWXNOR Truth Table

XNOR			LDCE		LDPE	
I0	I1	O	D	Q	D	Q
0	0	1	X	0	X	1
0	1	0	0	0	0	0
0	1	0	0	0	1	1
0	1	0	1	1	0	0
0	1	0	1	1	1	1
1	0	0	0	0	0	0
1	0	0	0	0	1	1
1	0	0	1	1	0	0
1	0	0	1	1	1	1
1	1	1	X	0	X	1

Xilinx FPGAs have 6 input 2 output look-up tables called LUT6_2. By taking XNOR's inputs, LDCE and LDPE's D inputs as inputs, and LDCE and LDPE's Q outputs as outputs, the given truth table can be implemented with a single LUT6_2. Cell design made with LUT6_2 is given in Fig. 6.

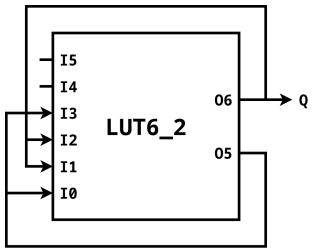


Fig. 6: LWXNORLUT design.

In the design shown in Fig. 6, I0 and I1 represent inputs of XNOR, I2 represents D input of LDCE, I3 represents D input of LDPE, O5 represents Q output of LDCE and O6 represents Q output of LDPE. O5 output is connected to I0 and I3 inputs and O6 output is connected to I1 and I2 inputs. According to these input and output representations, necessary LUT configurations are made to obtain the truth table seen in Table I. Thus, the LWXNOR design is implemented with a single LUT.

The working principle is similar to LWXNOR design. Assuming that in the initial state O5 is '0' and O6 is '1', I0 and I1 inputs will be (0,1). Thus, O5 will equal I2 and O6 will equal I3. As in the LWXNOR design, a metastability and race condition occurs here. And similarly, when the race is over, I0 and I1 will be equalized and O5 and O6 outputs will return to their initial state. This situation continues constantly. One of the unused inputs I4 or I5 can be used to start and stop the loop. Either output O5 or O6 can be sampled by a register to generate a random number.

3) Ring Oscillator with LDCE Latch (ROLDCE): Ring oscillators are designed by connecting an odd number of inverters in the form of a ring, and its output is sampled by a register and used for random number generation. While

designing in FPGA, LUT is generally used for inverters. If configured correctly, latches can also be used as inverters. A ring oscillator made using LDCE latch is given in Fig. 7. The benefit of such a design is that it avoids the use of LUTs and leaves space for other designs besides TRNGs.

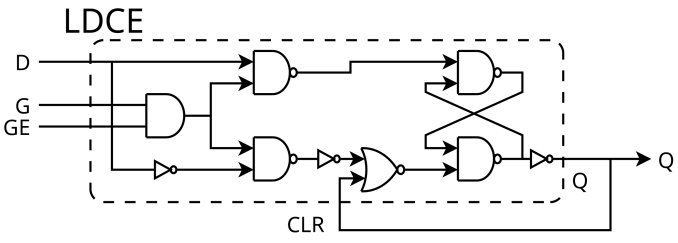


Fig. 7: ROLDCE Design.

G, GE, and D inputs shown in Fig. 7 are set to '1'. The Q output is connected to the CLR input. In this case, the reduced version of the ROLDCE design given in Fig. 7 is given in Fig. 8.

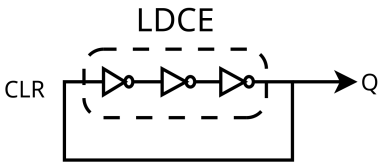


Fig. 8: Reduced ROLDCE Design.

As seen in Fig. 8, a ring containing three inverters can be obtained by making the G, GE, and D inputs '1'. If necessary, one of the G, GE, or D inputs should be set to '0' to stop the Oscillation. Random number generation can be done by sampling the output Q by a register.

4) Ring Oscillator with LDCE and LDPE Latches (ROLDCEPE): Similar to the ROLDCE design, a slightly larger ring can be obtained by combining LDCE and LDPE latch. In Fig. 9, the design made using LDCE and LDPE latch is given. Again, the benefit of such a design is avoiding the usage of LUTs.

In the design, the Q output of LDCE is connected to the PRE input of LDPE, and the Q output of LDPE is connected to the CLR input of LDCE. G and GE inputs are set to '1'. D input of LDCE is '1' and D input of LDPE is '0'. In this case, the reduced version of the design is given in Fig. 10.

By making a design as described, a ring oscillator containing 5 inverters can be obtained. Oscillation can be stopped by making one of the G or GE inputs '0', making D input of LDCE '0', or making D input of LDPE '1'. Random number generation can be made by registering one of Q outputs of LDCE or LDPE by a register.

B. TRNG Design

Multiple noise sources can be used in the TRNG design. The outputs of the noise sources are XORed, as shown in Fig. 11, and the XOR output is sampled by the clock of the register.

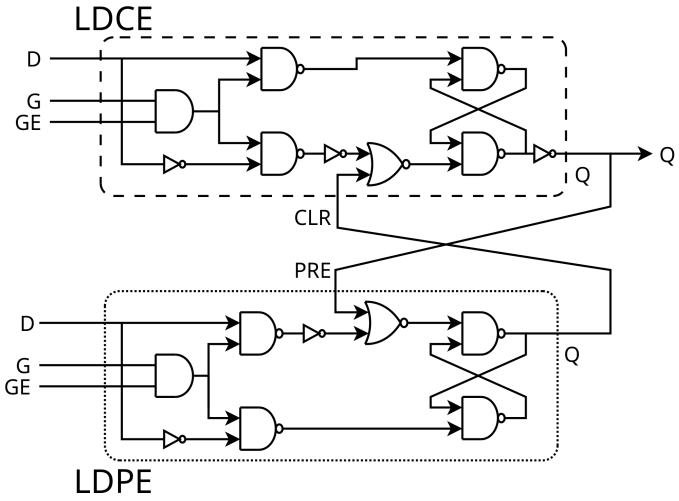


Fig. 9: ROLDCEPE Design.

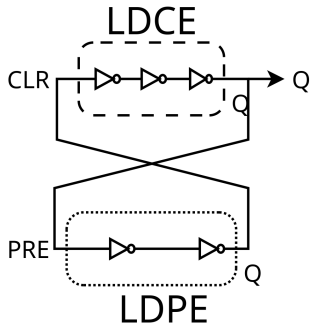


Fig. 10: Reduced ROLDCEPE Design.

To determine the optimal number of noise sources and the sampling frequency for the designs, some tests are performed by using the experimental setup in Section IV. 1, 4, 8, 12, 16, and 32 are chosen as the number of noise sources. 100, 125, 150, 175, 200, and 225 MHz are chosen as clock frequencies. The tests are performed at each noise source count and at each clock frequency.

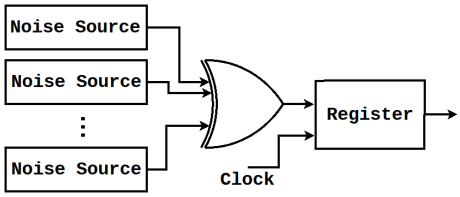
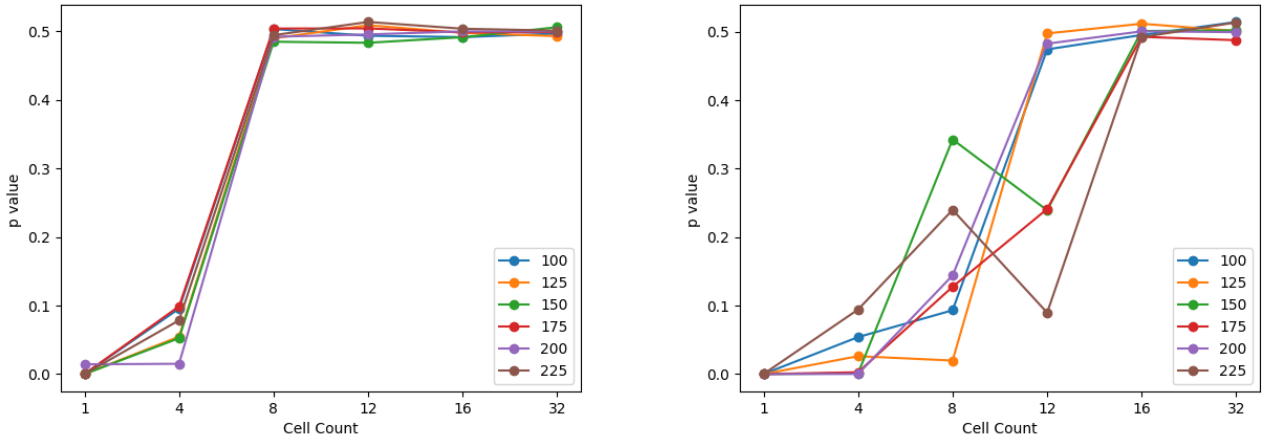


Fig. 11: Block Design of TRNG.

For testing, 100 million bits are generated from each design scenario. After that, the streams are divided into 10 Million bits. Then the divided streams are applied to NIST statistical tests, separately. For each statistical test, the p-values are averaged from 10 different test reports. Hence, the averages of p-values are obtained for a scenario. These processes are repeated for each scenario. In Fig. 12, p-value averages of



(a) LWXNOR

(b) LWXNORLUT

(c) ROLDCE

(d) ROLDCEPE

Fig. 12: P values averages of NonOverlapping Template Test.

Non-overlapping template test, which is one of the tests included in the NIST statistical test suite, are given.

As seen in Fig. 12a, the p-value for 1 cell is almost 0 at all frequencies. For 8 cells and more, it is fixed around 0.5. Therefore, the design should include 8 cells or more. Given the results of the other tests, it is chosen to use 12 cells because the design with only 8 cells sometimes fails for some statistical tests.

Similarly, 16 cell 225 MHz is chosen for LWXNORLUT by looking at Fig. 12b, 16 cells 225 MHz for ROLDCE by looking at figure Fig. 12c and 16 cells 200 MHz for ROLDCEPE by looking at Fig. 12d.

According to the obtained information, the maximum clock frequency for the proposed designs is 225MHz. The LUT/Flip-flop usages and throughput values are given in Table II with the values from the other implementations given in the literature for comparison. Furthermore, an efficiency metric (throughput/LUT count) is given for a fair comparison. Regarding the efficiency values, the proposed designs have the best results.

IV. Randomness Evaluation

In this paper, to evaluate the quality of the randomness for the proposed TRNG designs, three different statistical test platforms, NIST 800-22, NIST 800-90B, and AIS-20/31, are used.

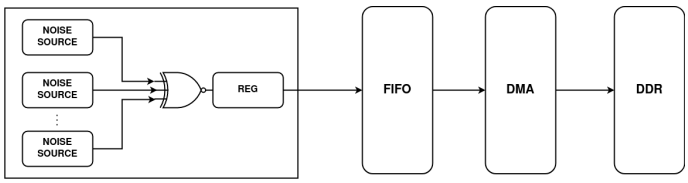


Fig. 13: Simple view of experiment environment

On a Xilinx 7-Series Development Board (Zedboard), which has the FPGA part named XC7Z020, the proposed TRNGs are successfully implemented. A data acquisition design with

TABLE II: Comparison the Proposed Design with Other TRNGs

Design	Area	Throughput (Mb/s)	Efficiency	FPGA
LWXNOR	15 LUTs + 25 FF	225	15	Xilinx XC7Z020
LWXNORLUT	19 LUTs + 1 FF	225	11.84	Xilinx XC7Z020
ROLDCE	3 LUTs + 17 FF	225	75	Xilinx XC7Z020
ROLDCEPE	3 LUTs + 33 FF	200	66.66	Xilinx XC7Z020
Metastability [15]	716 LUTs + 974 FFs	25	0.035	Xilinx XC7A35T
Self-Timed Rings [22]	14097 LUTs	10000	0.709	Xilinx XC7VX485T
Ring-oscillator [23]	528 LUTs + 177 FFs	6	0.011	Xilinx XC3S400A
Tetrahedral Oscillators [24]	62 LUTs	10	0.161	Xilinx XC7Z020

a maximum sample frequency of 225 MHz is used to capture the data. One million bits are gathered for each of the 100 bitstreams. The experimental setup shown in Fig. 13 is used to evaluate the implemented TRNG architecture and carry out the key procedures outlined in the flowchart seen in Fig. 14. The proposed TRNG doesn't require any initialization and it can provide random data as soon as it boots up. On a Linux machine, AIS-31, NIST 800-22, and NIST 800-90B tests are applied on the gathered 100 bitstreams. All the test results are successful for all the experiments.

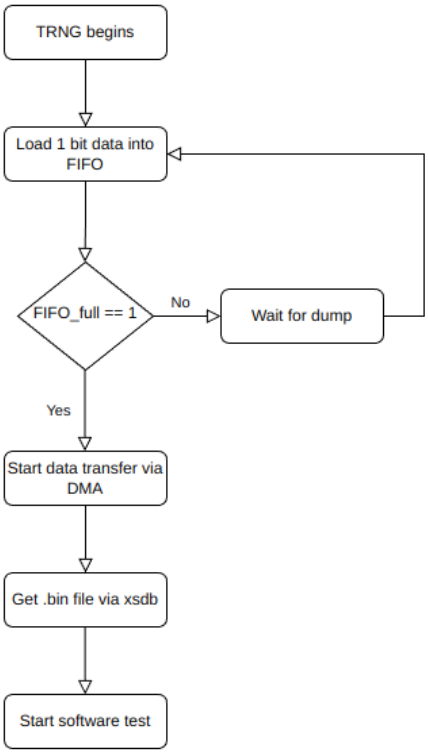


Fig. 14: Flow of the data gathering

The generated random numbers of the proposed design can be obtained by using `xsdb` software. DMA (Direct Memory Access) component has direct access to DDR. Hence, conveyed data, which is kept in DDR, can be obtained by `xsdb` software with JTAG interface. Therefore, a possible bottleneck during sending the random numbers from the FPGA board to the PC is overcome thanks to the speed of JTAG interface. An example of usage for `xsdb` software is depicted in Fig. 15.

```
./Command_Line
xsdb% mwr $DMA_ADDR 0
xsdb% while {[mrd -value $DMA_ADDR 1] == 0}
xsdb% mrd -bin -file $PATH $DMA_ADDR $LENGTH
```

Fig. 15: Sample view of the xsdb command line

A. AIS-31 Standard

A methodology for evaluating random number generators has been proposed by the German Federal Office for Information Security (AIS-20/31) [21], which should assist designers in better considering security aspects in their design and assist evaluators in rigorously evaluating the security of the entropy source during the certification process. Currently, AIS-20/31 compliance is required for any TRNGs focused on cryptographic applications that need a security certificate to be used in the European Union.

In this paper, all the procedures of the AIS-20/31 are employed to assess the statistical accuracy of the generated numbers. For estimating the entropy, all the tests in AIS-20/31 including T0 to T8 are used. In AIS-20/31 test suite, there is only one output that is used as a test result, which is whether it passed or not. All of the proposed designs pass all the tests in AIS-20/31 standard.

B. NIST 800-22 Statistical Test Suite

The Statistical Test Suite, which is established by the NIST, is one of the test environments to examine the randomness of the random numbers. A group of fifteen statistical tests makes up the NIST test suite. In each of these experiments, a certain feature of randomness is hypothetically quantified. If the P-value is larger than or equal to the target value a for a particular sample size n, the test is deemed to have been successful [20]. Table III provides the statistical test results for the proposed TRNGs. All the TRNGs pass all the statistical tests.

C. NIST 800-90B Tests

If all samples from a noise source are mutually independent and all samples have the same probability distribution, the samples are said to be independent and identically distributed (IID). Entropy estimate is made considerably easier by the IID assumption. Estimating entropy is more challenging when the

TABLE III: NIST Statistical Test Results

	LWXNOR		LWXNORLUT		ROLDCE		ROLDCEPE	
Tests	P-value	Pass Rate	P-value	Pass Rate	P-value	Pass Rate	P-value	Pass Rate
Frequency	0.779188	100/100	0.935716	99/100	0.739918	99/100	0.474986	100/100
BlockFrequency	0.191687	100/100	0.759756	100/100	0.719747	99/100	0.000216	98/100
CumulativeSums	0.332526	100/100	0.454076	100/100	0.524420	100/100	0.325274	100/100
Runs	0.017912	99/100	0.616305	98/100	0.224821	99/100	0.455937	100/100
LongestRun	0.834308	98/100	0.145326	98/100	0.071177	99/100	0.013569	99/100
Rank	0.004981	96/100	0.637119	100/100	0.055361	100/100	0.171867	98/100
FFT	0.401199	99/100	0.759756	99/100	0.350485	99/100	0.334538	100/100
NonOverlapping	0.554420	98/100	0.514124	98/100	0.350485	98/100	0.366918	98/100
Overlapping	0.816537	99/100	0.699313	96/100	0.040108	97/100	0.987896	98/100
Universal	0.637119	99/100	0.319084	100/100	0.637119	100/100	0.085587	98/100
ApproximateEntropy	0.657933	98/100	0.759756	99/100	0.115387	100/100	0.075719	98/100
Rand-Excur.	0.162606	61/62	0.554420	54/55	0.364146	65/65	0.275709	65/66
Rand-Variant	0.350485	61/62	0.474986	54/55	0.392456	65/65	0.232760	65/66
Serial	0.864188	98/100	0.416268	99/100	0.549667	100/100	0.724420	100/100
LinearComplexity	0.289667	100/100	0.911413	99/100	0.040108	99/100	0.455937	99/100

IID assumption is violated, which occurs when the samples are not equally or independently distributed (or both). There are several statistical tests in the IID section in NIST 800-90B standard that is intended to detect evidence that the samples are not IID; nevertheless, if no such evidence is discovered, it is considered that the samples are IID.

The ability to consistently provide random outputs is one of the fundamental needs of an entropy source. Determining the quantity of entropy produced per noise source sample is necessary to make sure that enough entropy is fed to the TRNG. Based on the submitter's examination of the noise source, the submitter must provide an entropy estimate for the outputs of the noise source. The designation for this estimate is $H_{submitter}$. Once the entropy estimation track has been established, a min-entropy estimate per sample is abbreviated $H_{original}$. The noise source's initial entropy estimate is calculated using the formula $H_I = min(H_{original}, nxH_{bitstring}, H_{submitter})$ [20].

For the proposed designs, obtained H_I and $H_{original}$ values with the software provided by NIST [27] are sufficient. As seen in Table IV, all the designs can be stated as IID.

TABLE IV: IID Test Results

	LWXNOR	LWXNORLUT	ROLDCE	ROLDCEPE
$H_{original}$	0.995782	0.994034	0.994465	0.996070
H_I	0.993238	0.994034	0.994465	0.995834

D. Independence from Constraints

One of the key benefits of proposed TRNGs is no need for design constraints which is necessary for other latch-based implementations to keep the delays in the close ranges. To validate that property, the same TRNG designs are implemented on three different FPGA parts. Then, the tests explained in Section IV are repeated for all the implementations. Finally, the test results for all the FPGA parts are successful.

V. Conclusion

In today's digital systems, information security issues are becoming increasingly important. The security of the entire system is directly impacted by the quality of a random number. In this paper, a novel approach for generating truly random numbers is introduced. According to the approach, instead of consuming necessary logic resources like LUTs for all designs, less-used latch primitives are employed. Therefore, the area of the proposed design is very small in terms of LUT usage. Although the Latch primitives introduced in the paper are given for Xilinx FPGAs, the approach can be easily extended to other FPGA vendors since they have similar primitives. Furthermore, the proposed designs do not require any timing and localization constraints since the operation depends on the delay ratio rather than absolute delay values. Therefore it is very easy to migrate the TRNG designs to another FPGA family of the same vendor. One of the other key benefits of the proposed TRNG designs is having no post-processing layer. Without post-processing, it passes all the statistical tests. In addition, the proposed designs have the best efficiency values regarding the other FPGA implementations given in the literature. Future research will examine the proposed designs' resistance to active and passive attacks.

References

[1] Ç. K. Koç, "About cryptographic engineering," in *Cryptographic Engineering*, Springer, Boston, MA, 2009, pp. 1-4.

[2] F. Tehranipoor, P. Wortman, N. Karimian, W. Yan, and J. A. Chandy, "DVFT: A Lightweight Solution for Power-Supply Noise-Based TRNG Using Dynamic Voltage Feedback Tuning System," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 26, no. 6, pp. 1084-1097, 2018.

[3] H. Guo, W. Tang, Y. Liu, and W. Wei, "Truly random number generation based on measurement of phase noise of a laser," *Phys. Rev. E*, vol. 81, p. 051137, 2010.

[4] X. Li, A. B. Cohen, T. E. Murphy, and R. Roy, "Scalable parallel physical random number generator based on a superluminescent LED," *Opt. Lett.*, vol. 36, pp. 1020–1022, 2011.

[5] Y. Ma, T. Chen, J. Lin, J. Yang, and J. Jing, "Entropy Estimation for ADC Sampling-Based True Random Number Generators," *IEEE Transactions on Information Forensics and Security*, vol. 14, no. 11, pp. 2887-2900, 2019.

[6] T. N. N. Kyaw and A. Tsuneda, "New sets of binary functions for generating orthogonal codes with negative auto-correlation based on Bernoulli chaotic map," in *Proc. Int. Conf. Inf. Commun. Technol. Convergence (ICTC)*, 2016, pp. 700–702.
[7] P. Li, et al., "Parallel optical random bit generator," *Optics Letters*, vol. 44, no. 10, pp. 2446-2449, 2019.
[8] T. McNichol, "Totally random," *Wired*, vol. 11, no. 8, 2003. [Online]. Available: http://www.wired.com/wired/archive/11.08/random.html.
[9] S.-K. Yoo, D. Karakoyunlu, B. Birand, and B. Sunar, "Improving the robustness of ring oscillator TRNGs," *ACM Trans. Reconfigur. Technol. Syst.*, vol. 3, no. 2, pp. 9:1–30, 2010.
[10] X. Li, et al., "Jitter-based adaptive true random number generation for FPGAs in the cloud," in *2020 International Conference on Field-Programmable Technology (ICFPT)*, IEEE, 2020.
[11] D. Kinniment and E. G. Chester, "Design of an on–chip random number generator using metastability," *IEICE Electronics Express*, vol. 15, no. 10, 2018.
[12] C. Tokunaga, D. Blaauw, and T. Mudge, "True Random Number Generator with a Metastability-Based Quality Control," in *2007 IEEE International Solid-State Circuits Conference. Digest of Technical Papers*, 2007, pp. 404-611, doi: 10.1109/ISSCC.2007.373465.
[13] N. Torii, et al., "Evaluation of latch-based physical random number generator implementation on 40 nm ASICs," *Proc. TrustED*, 2016, p. 23.
[14] C. Li, Q. Wang, J. Jiang, and N. Guan, "A metastability-based true random number generator on FPGA," *IEEE 12th International Conference on ASIC (ASICON)*, 2017, pp. 738-741, doi: 10.1109/ASICON.2017.8252581.
[15] N. Fujieda and S. Ichikawa, "A latch-latch composition of metastability-based true random number generator for Xilinx FPGAs," *IEICE Electronics Express*, vol. 15, no. 10, 2018.
[16] G. D. P. Stanchieri et al., "A true random number generator architecture based on a reduced number of FPGA primitives," *AEU-International Journal of Electronics and Communications*, vol. 105, pp. 15-23, 2019.
[17] L. Fan, et al., "A true random number generator based on meta-stable state," *IEICE Electronics Express*, vol. 15, no. 1, 2018.
[18] Xilinx, "Vivado Design Suite 7 Series FPGA and Zynq-7000 SoC Libraries Guide (UG953)," 2022.
[19] L. E. Bassham III, et al., "A statistical test suite for random and pseudorandom number generators for cryptographic applications," *SP 800-22 rev. 1a*, National Institute of Standards & Technology, 2010.
[20] M. S. Turan, et al., "Recommendation for the entropy sources used for random bit generation," *NIST Special Publication 800.90B*, 2018.
[21] W. Killmann and W. Schindler, "A proposal for: Functionality classes for random number generators, version 2.0," 2011.
[22] Z. Lu, et al., "Efficient FPGA implementation of high-speed true random number generator," *Review of Scientific Instruments*, vol. 92, no. 2, 2021.
[23] N. N. Anandakumar, S. K. Sanadhya, and M. S. Hashmi, "FPGA-based true random number generation using programmable delays in oscillator-rings," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 67, no. 3, pp. 570-574, 2019.
[24] R. Günay and S. Ergün, "IC random number generator exploiting two simultaneous metastable events of tetrahedral oscillators," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 67, no. 9, pp. 1634-1638, 2020.
[25] M. A. Prada-Delgado, C. Martínez-Gómez, and I. Baturone, "Auto-calibrated ring oscillator TRNG based on jitter accumulation," in *2020 IEEE International Symposium on Circuits and Systems (ISCAS)*, 2020.
[26] P. Z. Wieczorek and K. Gołofit, "True random number generator based on flip-flop resolve time instability boosted by random chaotic source," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 65, no. 4, pp. 1279-1292, 2017.
[27] The SP800-90B_EntropyAssessment C++ package implements the min-entropy assessment methods included in Special Publication 800-90B. [Online]. Available: https://github.com/usnistgov/SP800-90B_EntropyAssessment (accessed Oct 20, 2022).